ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

Project Report

CI/CD Pipeline for an Admin Dashboard Monorepo

— **Program:** Big Data and Internet of Things
Academic Year: 2025 – 2026

Authors:
Karrouach Ansar

Supervised by:

January 3, 2026

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

Contents

1. Introduction

When deploying software manually, even experienced developers make mistakes. A forgotten environment variable, an inconsistent configuration, or a missed test can bring down a production system. As the Admin Dashboard application grew from a simple prototype to a full-featured e-commerce administration platform, these manual deployment issues became increasingly costly both in time spent debugging and in user trust lost during outages.

This project addresses these challenges by implementing a complete CI/CD (Continuous Integration and Continuous Deployment) pipeline. The goal is straightforward: every code change pushed to the repository should automatically go through testing, quality analysis, containerization, and deployment without any manual intervention. If something fails, the team should know immediately.

The report is structured as follows. We begin with an overview of the application and its technical stack, then examine the specific problems that motivated this work. The proposed solution section presents the architecture and tools selected. We then describe the methodology used to build the pipeline iteratively, followed by detailed design decisions and tool comparisons. Finally, we present the implementation details and evaluate the results achieved.

2. Project Overview

Before discussing the CI/CD pipeline, it is essential to understand the application it serves and the technical constraints that shaped our decisions.

2.1 Application Description

The *Admin Dashboard* is a full-stack web application designed for business users to manage core entities such as products, categories, sizes, colors, orders, and user accounts. The application serves as an administrative interface for an e-commerce platform, providing functionalities for inventory management, order tracking, and user administration.

The codebase is organized as a **monorepo**, which means both the backend and frontend reside in a single repository. This architecture simplifies version control and ensures that related changes across both modules can be committed and deployed together.

2.2 Technical Stack

The application comprises the following components:

- **Backend (API)**: A Spring Boot application (Java 17) exposing RESTful endpoints for all business operations.

- **Frontend**: A Next.js application (React/Node.js) that consumes the API and provides the user interface.
- **Database**: PostgreSQL hosted on Google Cloud SQL for persistent data storage.
- **File Storage**: AWS S3 for storing uploaded files such as product images and documents.

3. Problem Statement

With a clear understanding of the application, we can now examine the deployment challenges that this project aims to solve.

3.1 Context

Before this project, deployments were performed manually or with minimal automation. Developers would build the application locally, create Docker images, and manually execute deployment commands. This approach worked for initial development but became increasingly problematic as the project evolved and multiple team members needed to contribute.

3.2 Identified Problems

The manual deployment process led to several recurring issues that affected both productivity and reliability.

Human errors were the most frequent problem. Developers occasionally forgot to set environment variables correctly, used inconsistent configuration across environments, or deployed with missing secrets. These errors often resulted in application failures that were difficult to diagnose because the root cause was configuration-related rather than code-related.

Version mismatches occurred when the API and frontend were deployed at incompatible versions. Since both modules evolve together, deploying an older frontend with a newer API (or vice versa) could cause runtime failures. Without automated coordination, ensuring version consistency required manual tracking and careful coordination.

Inconsistent quality resulted from the absence of systematic checks. Code reviews happened informally, and tests were not always executed before deployment. This allowed defects and technical debt to accumulate, sometimes reaching production before being detected.

Poor traceability made debugging production issues challenging. When a problem occurred, determining which exact code version was running required manual investigation through deployment logs and container registries.

Slow delivery was the cumulative effect of all these issues. Manual steps increased lead time from commit to production, reducing the team's ability to iterate quickly and respond to feedback.

3.3 Objective

The primary goal of this project is to implement an automated, end-to-end CI/CD pipeline that addresses each of these problems. The pipeline should provide fast feedback through automated testing, enforce code quality standards through static analysis, produce reproducible containerized builds with clear version tags, deploy reliably to a cloud platform, and notify the team immediately of build status.

4. Proposed Solution

Having identified the problems, we designed a solution that addresses each one systematically while remaining cost-effective and maintainable.

4.1 Solution Overview

The solution is a fully automated CI/CD chain hosted on Google Cloud Platform (GCP). The architecture was designed with cost efficiency in mind, using self-hosted tools where possible while leveraging managed cloud services for deployment and data storage.

At the core of the solution is **Jenkins**, running on a Compute Engine virtual machine. Jenkins orchestrates the entire pipeline, from code checkout through deployment. It was chosen for its flexibility, extensive plugin ecosystem, and the learning opportunities it provides in understanding CI/CD internals.

Code quality is enforced through **SonarQube**, also hosted on a dedicated Compute Engine VM. SonarQube performs static code analysis on every build and enforces a Quality Gate that must pass before deployment can proceed. This ensures that code meeting minimum quality standards reaches production.

Both application modules are packaged as **Docker** containers, ensuring consistent behavior across development, testing, and production environments. The container images are stored on **Docker Hub**, tagged with the Jenkins build number for traceability.

The applications are deployed to **Google Cloud Run**, a serverless container platform that provides automatic scaling, high availability, and pay-per-use pricing. The API connects to a managed **Cloud SQL** PostgreSQL database, while sensitive configuration is stored in **Secret Manager** and injected at runtime.

4.2 Automation and Notifications

The pipeline is triggered automatically through a **GitHub webhook** configured on the repository. Every push to the main branch initiates a new pipeline run without any manual intervention.

At the end of each run, Jenkins sends an **email notification** to the team. Success emails confirm the deployment and include direct links to the deployed services along with build duration. Failure emails alert the team immediately and provide a link to the console output for debugging.

5. Methodology

With the solution architecture defined, we needed a systematic approach to build it. The project was executed using an iterative methodology, allowing continuous validation and refinement at each stage. Rather than attempting to build the entire pipeline at once, we broke the work into five distinct phases, validating each component before moving to the next. This approach minimized risk and allowed us to learn from each phase before proceeding.

5.1 Phase 1: Infrastructure Provisioning

The first phase focused on establishing the foundational infrastructure. We created two Compute Engine VMs in the `us-central1` region: one for Jenkins and one for SonarQube. Both VMs use the `e2-medium` machine type, which provides a good balance between cost and performance for our workload.

On the Jenkins VM, we installed Docker, Node.js, Maven, and the Google Cloud CLI. The SonarQube VM required PostgreSQL for its internal database, along with the SonarQube server itself. Network firewall rules were configured to allow access to Jenkins (port 8080) and SonarQube (port 9000) while restricting unnecessary exposure.

5.2 Phase 2: Pipeline Development

With infrastructure in place, we developed the Jenkins pipeline using a declarative Jenkinsfile. The pipeline was built incrementally, starting with a simple checkout stage and gradually adding more functionality. The GitHub webhook was configured to trigger builds automatically on every push to the main branch.

5.3 Phase 3: Quality Integration

The third phase integrated SonarQube into the pipeline. We configured Jenkins to communicate with the SonarQube server using a dedicated authentication token. A Quality Gate stage was added that queries SonarQube after analysis completes and fails the pipeline if quality thresholds are not met.

5.4 Phase 4: Containerization and Deployment

Dockerfiles were created for both the API and frontend modules. We used multi-stage builds to produce smaller, more secure images by separating build-time dependencies from runtime artifacts. Cloud Run services were configured with secrets injected from Secret Manager, implementing sequential deployment (API first, then frontend) to avoid race conditions.

5.5 Phase 5: Optimization

The final phase focused on performance optimization. We enabled Docker layer caching, removed redundant build steps, and parallelized the test stage. These optimizations reduced total pipeline time from approximately 39 minutes to approximately 13 minutes.

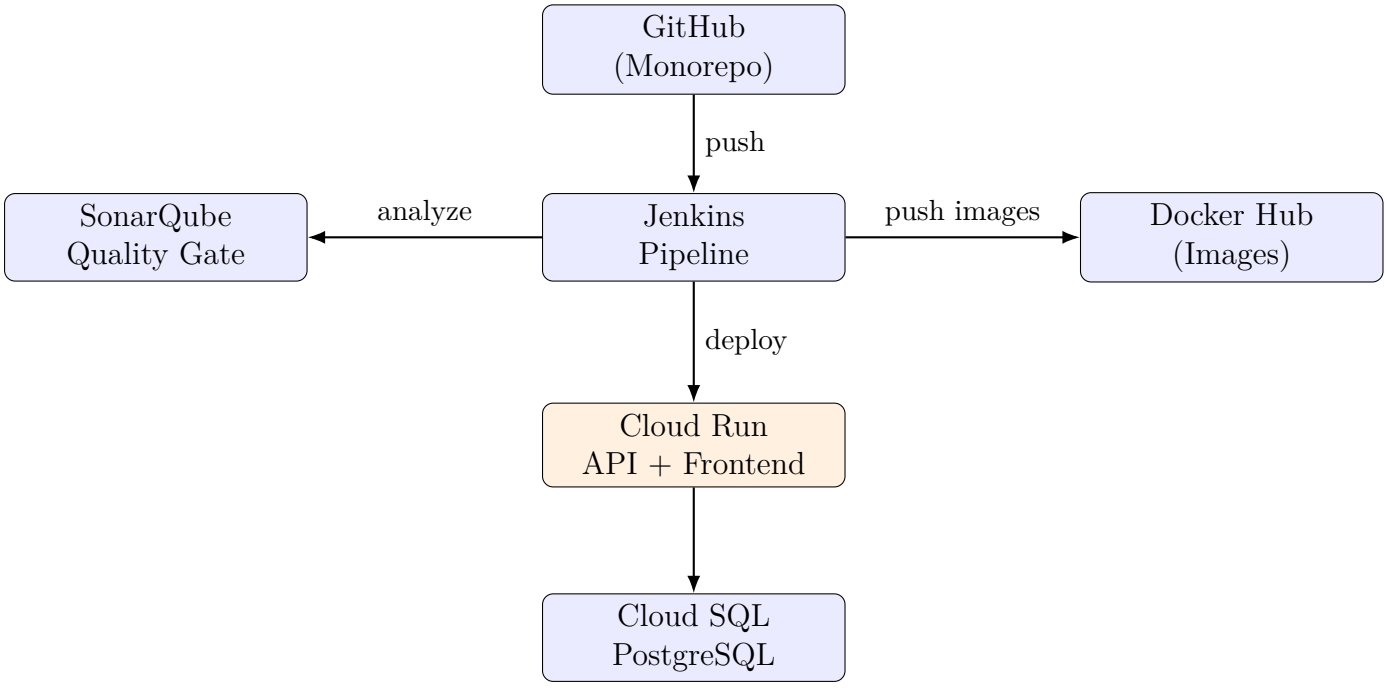
5.6 Validation Criteria

Throughout the project, we validated our progress against several criteria: the pipeline must execute end-to-end without manual intervention, Cloud Run services must be accessible and functional, the SonarQube Quality Gate must be enforced, and execution time should remain under 15 minutes.

6. Design

6.1 Logical Architecture

The diagram below illustrates the flow from code commit to production deployment:



When a developer pushes code to GitHub, the webhook triggers Jenkins. Jenkins then orchestrates all subsequent steps: running tests, sending code to SonarQube for analysis, building and pushing Docker images, and finally deploying to Cloud Run.

6.2 Pipeline Stages

The pipeline consists of nine sequential stages, with some parallelization where possible.

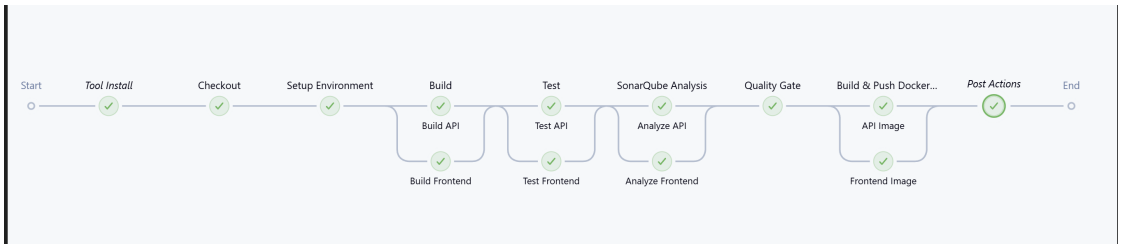


Figure 1: Jenkins pipeline view with stage status.

Checkout clones the repository from GitHub, ensuring the pipeline always works with the latest code.

Setup Environment injects environment files (`.env`) from Jenkins credentials into the workspace. This ensures sensitive configuration is never stored in the repository.

Test executes both API tests (using Maven/JUnit) and frontend tests (using Jest/npm) in parallel. Running tests in parallel significantly reduces feedback time.

SonarQube Analysis sends the code to the SonarQube server for static analysis. The server evaluates metrics including code coverage, duplications, bugs, vulnerabilities, and code smells.

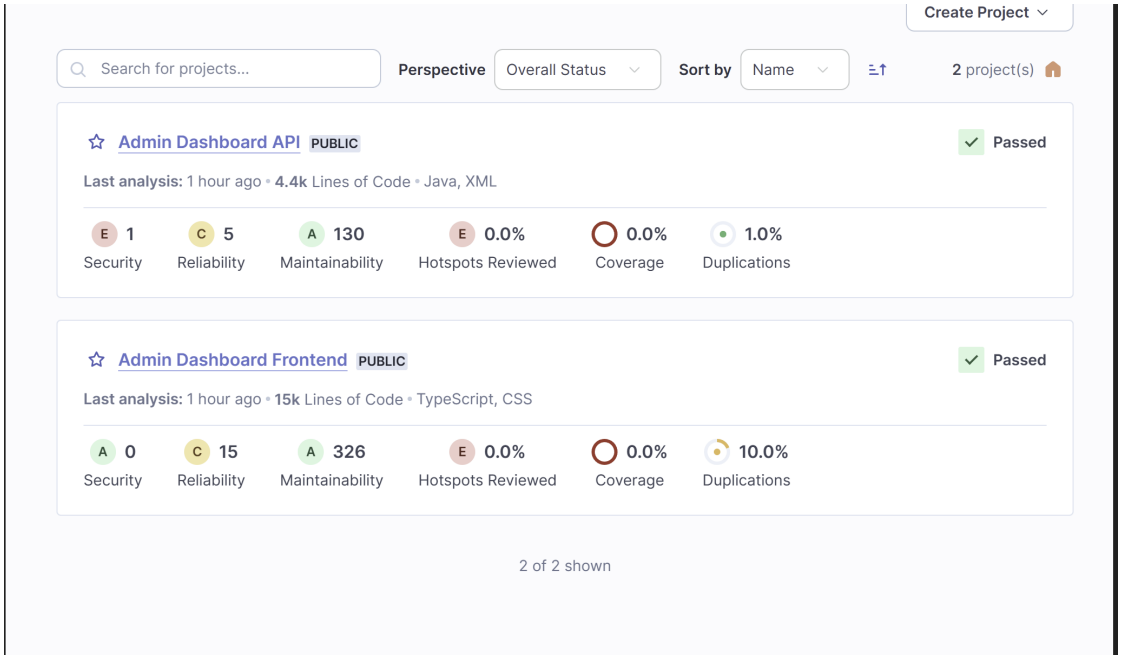


Figure 2: SonarQube dashboard showing analysis results and Quality Gate status.

Quality Gate queries SonarQube for the analysis verdict. If the Quality Gate fails

(e.g., too many bugs or insufficient coverage), the pipeline is aborted before any deployment occurs.

Build Docker Images creates container images for both the API and frontend. Each image is tagged with the Jenkins build number for traceability.

Push Images uploads the newly built images to Docker Hub with both a versioned tag (e.g., `api:42`) and the `latest` tag.

Deploy deploys the services to Cloud Run. The API is deployed first, followed by a 30-second delay, then the frontend. This sequential approach prevents overwhelming GCP APIs and ensures the API is available when the frontend starts.

Notify sends an email with the build status, including links to deployed services and build duration.

6.3 Deployment Strategy

The deployment strategy was designed with reliability as the primary concern, accepting slightly longer deployment times in exchange for stability.

The API service is always deployed before the frontend. This ordering ensures that when the new frontend version starts, the API endpoints it depends on are already available. A 30-second delay between deployments prevents overwhelming the GCP APIs and allows the API service to fully initialize before the frontend deployment begins.

Every Docker image is tagged with the Jenkins build number (e.g., `api:42`), creating a complete audit trail. If a deployment introduces a bug, rolling back is as simple as redeploying the previous tagged image. This traceability was one of the key requirements identified in our problem statement.

7. Tool and Method Comparison

The design decisions presented above required careful tool selection. Each choice involved trade-offs between simplicity, cost, flexibility, and learning value. This section documents the comparisons we conducted and the reasoning behind our final selections.

7.1 CI/CD Orchestration: Jenkins vs GitHub Actions

The choice of CI/CD orchestrator was between Jenkins, a mature self-hosted solution, and GitHub Actions, a modern SaaS offering integrated with GitHub.

Criteria	Jenkins	GitHub Actions
Hosting	Self-hosted (full control)	Managed SaaS by GitHub
Cost model	VM infrastructure cost only	Free tier with minute limits; pay-as-you-go beyond
Flexibility	Extensive plugin ecosystem; custom agents	Good, but confined to GitHub ecosystem

Maintenance	Requires updates, backups, monitoring	Minimal (managed service)
Learning value	Deep exposure to CI/CD internals	Easier onboarding, less ops experience

We selected **Jenkins** for this project. While GitHub Actions would have been simpler to set up, Jenkins provided greater flexibility and deeper learning opportunities. Managing our own CI/CD server gave us hands-on experience with infrastructure, troubleshooting, and system administration, skills that are valuable in professional DevOps roles.

7.2 Code Quality: SonarQube vs Cloud Alternatives

For static code analysis, we compared SonarQube (self-hosted) with cloud-based alternatives such as CodeClimate and Codacy.

SonarQube's Community Edition is free and provides comprehensive analysis including bug detection, vulnerability scanning, code smell identification, and coverage tracking. It supports multiple languages and offers powerful Quality Gates that can enforce minimum standards. Cloud alternatives offer easier setup and no infrastructure to maintain, but they come with recurring subscription costs and less customization.

We chose **SonarQube** for its cost efficiency (no subscription fees), depth of analysis, and the educational value of managing the tool ourselves.

7.3 Deployment Platform: Cloud Run vs Kubernetes

For hosting the deployed applications, we evaluated Google Cloud Run and Google Kubernetes Engine (GKE).

Cloud Run provides a serverless container platform with automatic scaling, pay-per-use pricing, and minimal operational overhead. Applications scale to zero when not in use, making it cost-effective for variable workloads. Kubernetes offers greater flexibility and control but requires cluster management, networking configuration, monitoring setup, and ongoing maintenance.

We selected **Cloud Run** for its simplicity and fast time-to-production. For a project of this scope, Cloud Run's managed infrastructure allowed us to focus on the CI/CD pipeline rather than cluster operations.

8. Implementation

With the design finalized and tools selected, we proceeded to implement the pipeline. This section focuses on the practical aspects: how notifications work, how deployments are configured, and the security measures we put in place.

8.1 Email Notifications

The final stage of every pipeline run is notification. Automated emails ensure the team stays informed without constantly monitoring the Jenkins dashboard a significant improvement over manually checking deployment status.

When a pipeline succeeds, the email includes everything a developer needs: direct links to both deployed services (API and frontend), the total build duration, and a link to the full build log. Team members can immediately click through to verify the deployment or share the URLs with stakeholders for review.

When a pipeline fails, speed of diagnosis is critical. Failure emails clearly indicate which stage failed (tests, quality gate, build, or deployment) and include a direct link to the console output. This allows developers to jump straight to the relevant logs without navigating through the Jenkins interface.

8.2 Deployment Configuration

The deployment to Cloud Run uses the Google Cloud CLI with secrets injected from Secret Manager. This approach ensures that sensitive configuration (database credentials, JWT secrets, API keys) is never stored in the repository or exposed in logs.

```
# Deploy API service with secrets
gcloud run deploy admin-api \
  --image docker.io/<username/api:${BUILD_NUMBER} \
  --region us-central1 \
  --set-secrets DATABASE_URL=DATABASE_URL:latest \
  --quiet

# Deploy Frontend service
gcloud run deploy admin-frontend \
  --image docker.io/<username>/frontend:${BUILD_NUMBER} \
  --region us-central1 \
  --quiet
```

8.3 Security and Best Practices

Security was a priority throughout the implementation. All secrets are stored in Google Secret Manager and injected at runtime, never appearing in source code or build logs. Docker images are tagged with build numbers for complete traceability, allowing us to identify exactly which code version is running and to roll back if needed. The sequential deployment strategy (API first, then frontend with a delay) prevents race conditions and ensures service availability.

9. Results

After completing the implementation, we evaluated the pipeline against the objectives defined at the start of the project. The results demonstrate that all initial goals were achieved, with some outcomes exceeding expectations.

Full automation eliminates the manual steps that previously caused errors and delays. Developers simply push their code, and the pipeline handles checkout, testing, analysis, building, and deployment without any intervention. This has significantly reduced the cognitive load on the team and freed time for actual development work.

Quality enforcement through the SonarQube Quality Gate ensures that only code meeting defined standards reaches production. Since implementing the pipeline, no deployments have occurred with failing quality checks. This has improved overall code quality and reduced the number of bugs discovered in production.

Performance optimization reduced pipeline duration from approximately 39 minutes to approximately 13 minutes. This improvement came from three main changes: enabling Docker layer caching (which avoids rebuilding unchanged layers), removing redundant build steps (the Docker build process now handles compilation), and parallelizing tests (API and frontend tests run simultaneously). Faster pipelines mean faster feedback, which improves developer productivity.

Complete traceability links every deployment to a specific build number and Docker image tag. When investigating a production issue, we can immediately identify which code version is running by checking the image tag. If needed, rolling back to a previous version is as simple as redeploying an earlier tagged image.

Immediate visibility through email notifications keeps the team informed without requiring constant monitoring. Team members receive notifications within seconds of build completion, allowing them to verify deployments or investigate failures promptly.

10. Conclusion

This project set out to solve a concrete problem: manual deployments were causing errors, delays, and frustration. The solution is a fully automated CI/CD pipeline integrating Jenkins, SonarQube, Docker, and Google Cloud Run that addresses each of the initial problems identified. Manual errors are eliminated through automation. Quality is enforced through the SonarQube Quality Gate. Traceability is achieved through versioned Docker images. And delivery speed improved from unpredictable manual processes to consistent 13-minute automated pipelines.

The pipeline demonstrates that modern DevOps practices can be implemented effectively even with self-hosted tools. While cloud-based CI/CD services offer convenience, self-hosting Jenkins and SonarQube provided valuable learning experiences and complete control over our infrastructure. The cost-conscious approach is using appropriate VM sizes, leveraging free tiers, and optimizing resource usage shows that professional-grade CI/CD can be achieved without excessive cloud spending.

Looking forward, several improvements could enhance the pipeline further. Migrating from Docker Hub to Google Artifact Registry would reduce network egress costs and improve image pull times for Cloud Run deployments. Adding end-to-end (E2E) tests using tools like Cypress or Playwright would provide broader test coverage beyond unit and integration tests. Implementing blue-green or canary deployment strategies would enable zero-downtime releases and safer rollouts. Finally, setting up cost monitoring and budget alerts would help control cloud spending as usage grows.

The skills and knowledge gained from this project infrastructure management, pipeline design, quality gate configuration, containerization, and cloud deployment that are directly applicable to professional software development environments. The pipeline serves not only as a working solution for the Admin Dashboard application but also as a template and learning resource for future projects.